

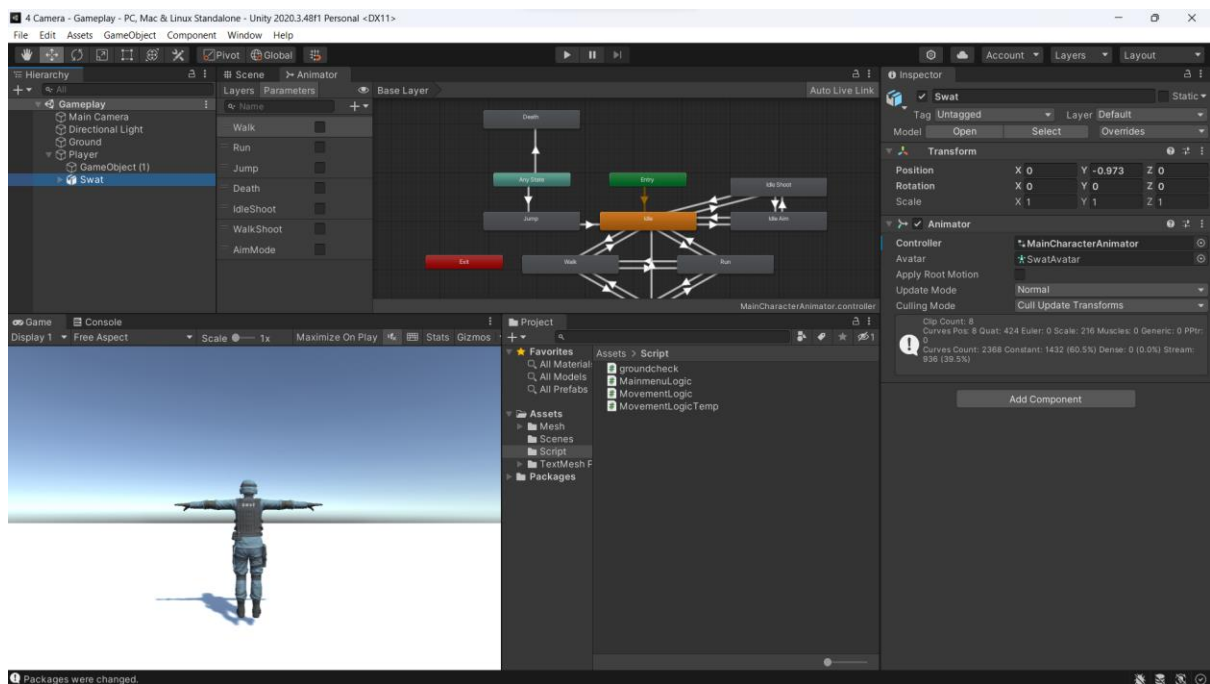
MODUL 4

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

Camera & Cinemachine

Pada Modul ini, Mahasiswa akan mempelajari tentang konfigurasi kamera didalam unity engine, Dan juga akan mempelajari Plugin **Cinemachine** yang berisikan banyak fitur kamera yang banyak diimplementasikan di industry game development. Berikut adalah Langkah Langkah melakukan konfigurasi kamera dalam unity engine.

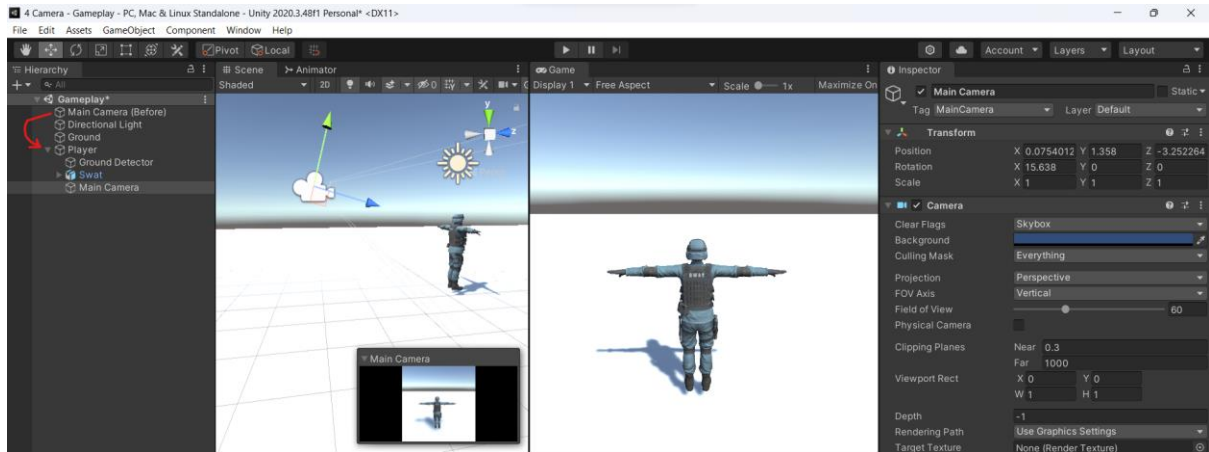
1. Buka Project Unity Modul 3 Anda (Animation & Animator FSM)



2. Konfigurasi Kamera Bawaan Unity

Konfigurasi paling sederhana untuk membuat kamera **TPS (Third Person Shooter)**, bisa menggunakan kamera bawaan unity yang sudah tergenerate sejak awal pembuatan Scene dilakukan.

Caranya adalah posisikan kamera hingga terlihat seperti gambar di bawah ini. Dan pindah posisi kamera ke menjadi **Child** GameObject **Player**.

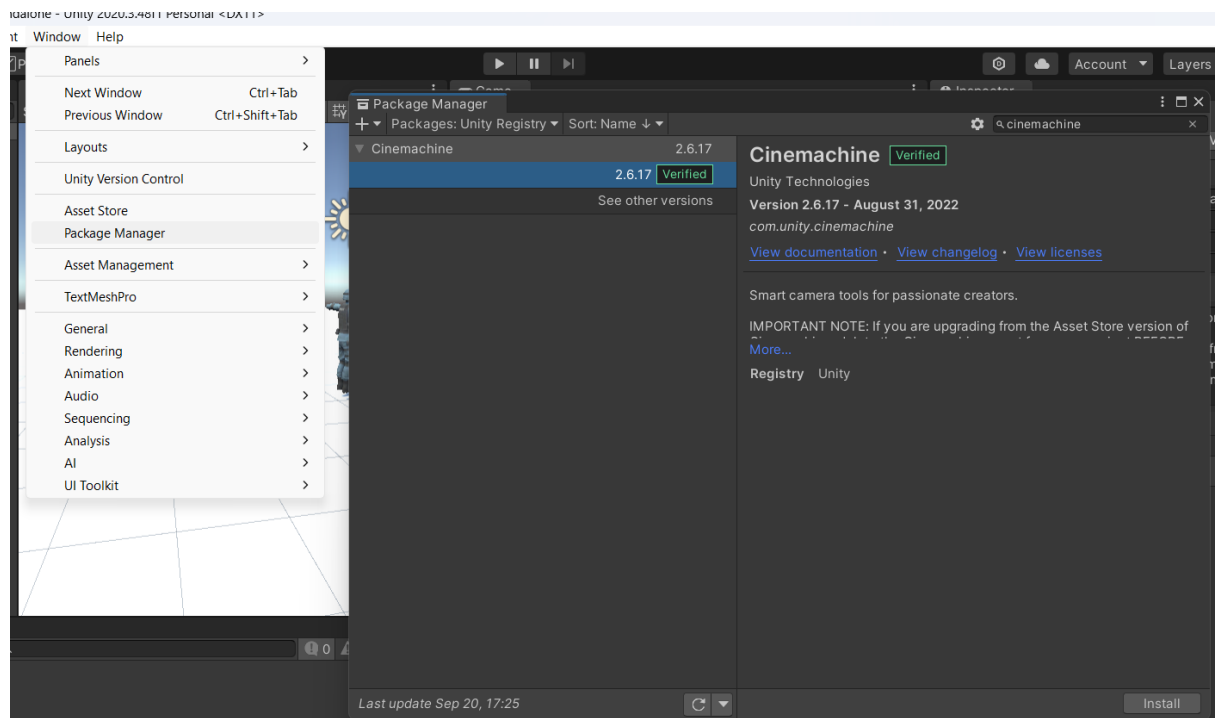


Jika sudah masuk kedalam **Play Mode** dan amati perbedaanya, jika kamera bergerak mengikuti posisi karakter berjalan, maka konfigurasi sudah benar, namun jika belum, lakukan pengecekan ulang!

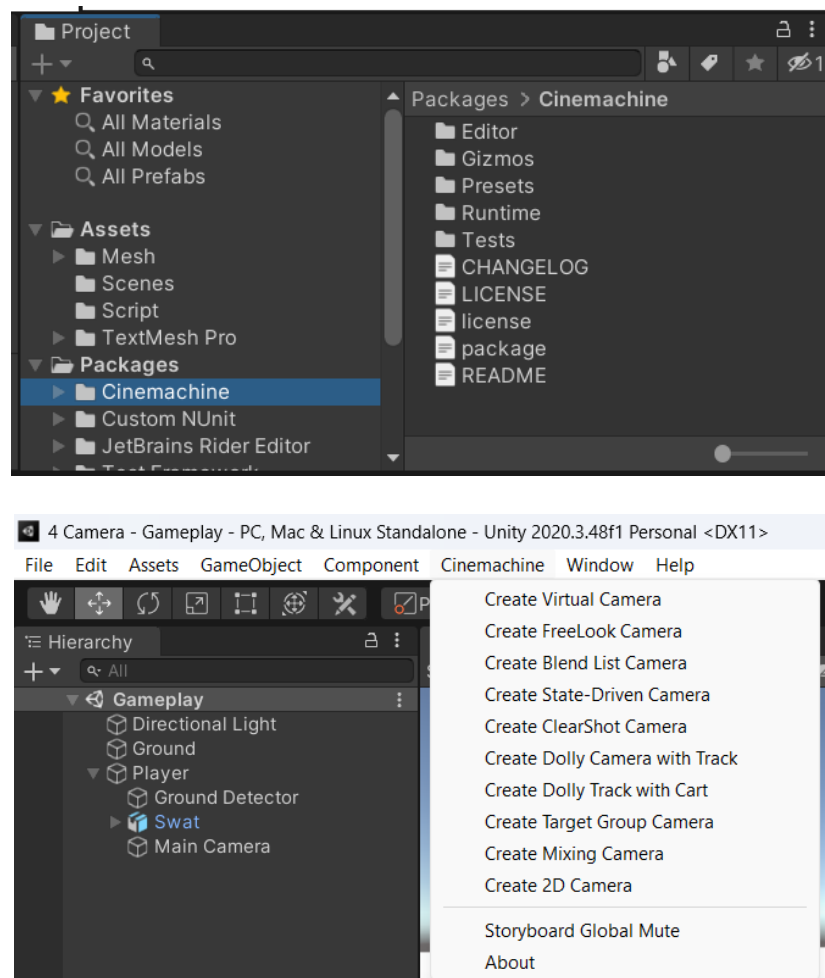
3. Instalasi Plugin Cinemachine

Selanjutnya adalah melakukan konfigurasi kamera menggunakan plugin **Cinemachine**. Langkah yang harus dilakukan adalah **download & Install** plugin dari **Package Manager** Unity. Berikut adalah Langkah download Cinemachine dari Unity **Package Manager**.

Window – Package Manager – Packages : Unity Registry – Search : Cinemachine – Seleksi Package Cinemachine – Download/Install



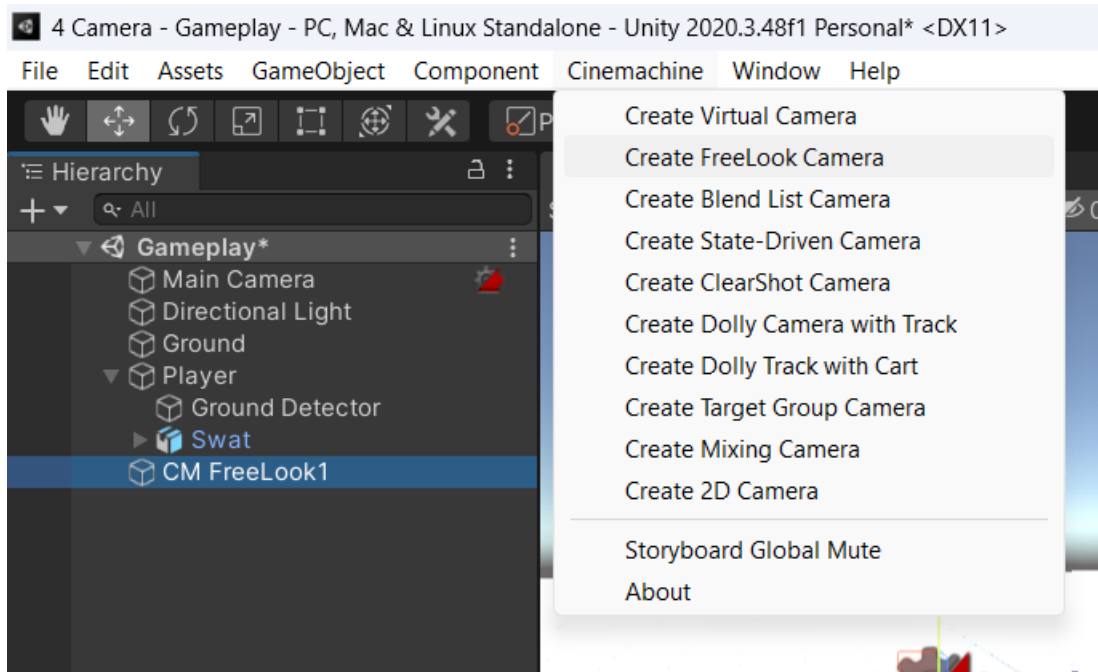
jika instalasi Package Cinemachine berhasil, maka akan terlihat folder Cinemachine pada folder Package di Window Project seperti di bawah ini, dan juga ada opsi **Cinemachine** diantara **Component** dan **Window**. jika masih belum ada, lakukan instalasi ulang!



4. Create Cinemachine Camera.

Setelah proses Instalasi Cinemachine selesai, proses selanjutnya adalah Create kamera Cinemachine agar dapat dioprasikan kedalam game.

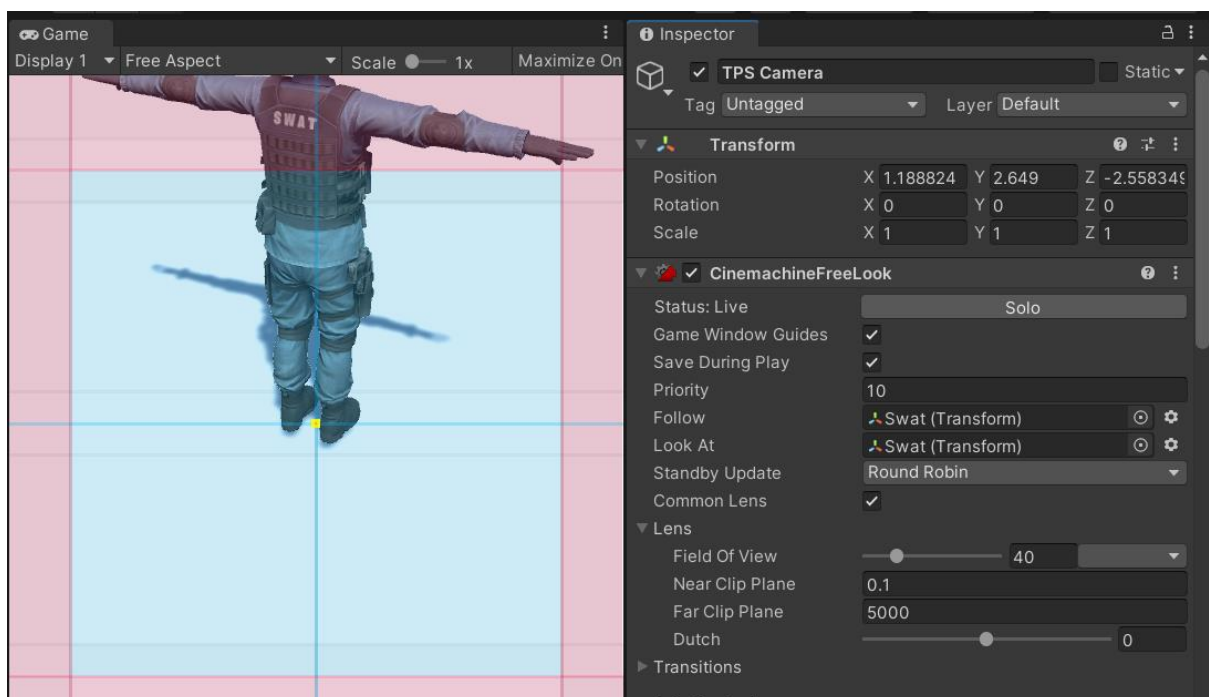
- Langkah pertama adalah mengembalikan **Main Camera** yang sebelumnya **Child Player** menjadi GameObject yang berdiri sendiri.
- Selanjutnya adalah **Create FreeLook Camera Cinemachine**.



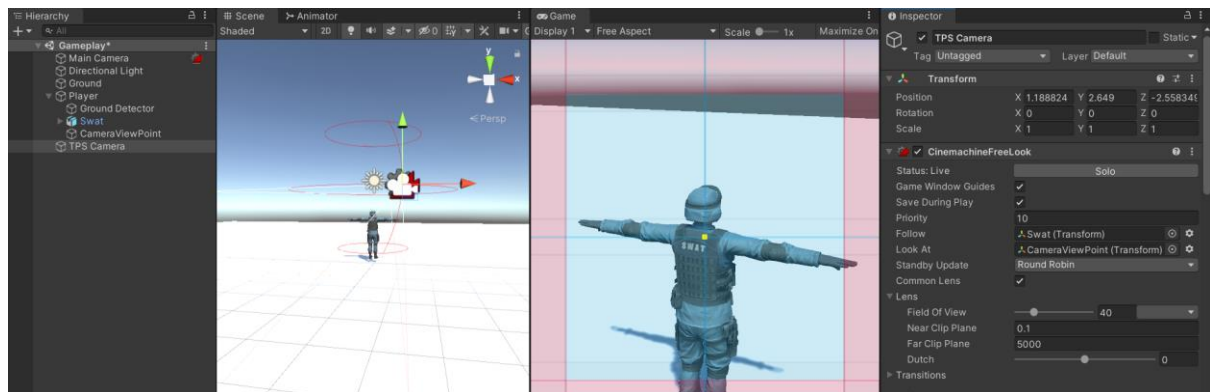
Rename GameObject **“CM FreeLook1”** menjadi **“TPS Camera”**.

5. Konfigurasi Camera Cinemachine (View Point)

Selanjutnya adalah Konfigurasi camera Cinemachine, konfigurasi dapat dilakukan dengan seleksi GameObject **TPS Camera**, dan melakukan beberapa konfigurasi pada inspector. Langkah pertama adalah Drag and drop GameObject Swat ke kolom Follow dan Look at. Jika konfigurasi sudah benar, tampilan Camera akan terlihat seperti gambar dibawah ini.



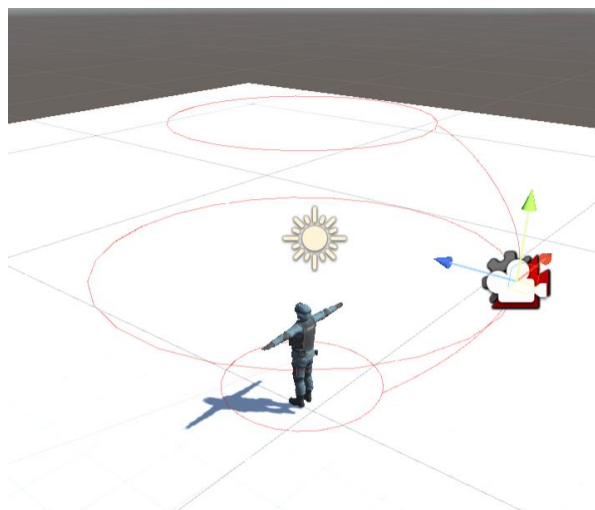
Terlihat focus kamera berada pada kaki karakter, dan karena pada Game biasanya focus kamera berada pada badan, oleh karena itu, Langkah selanjutnya adalah membuat **EmptyGameObject** dengan nama **CameraViewPoint** lalu diposisikan di badan karakter. Setelah itu drag ke kolom **Look At**.



Jika konfigurasi benar, maka camera akan berpindah focus ke badan karakter. Jika masih belum, lakukan pengecekan ulang!

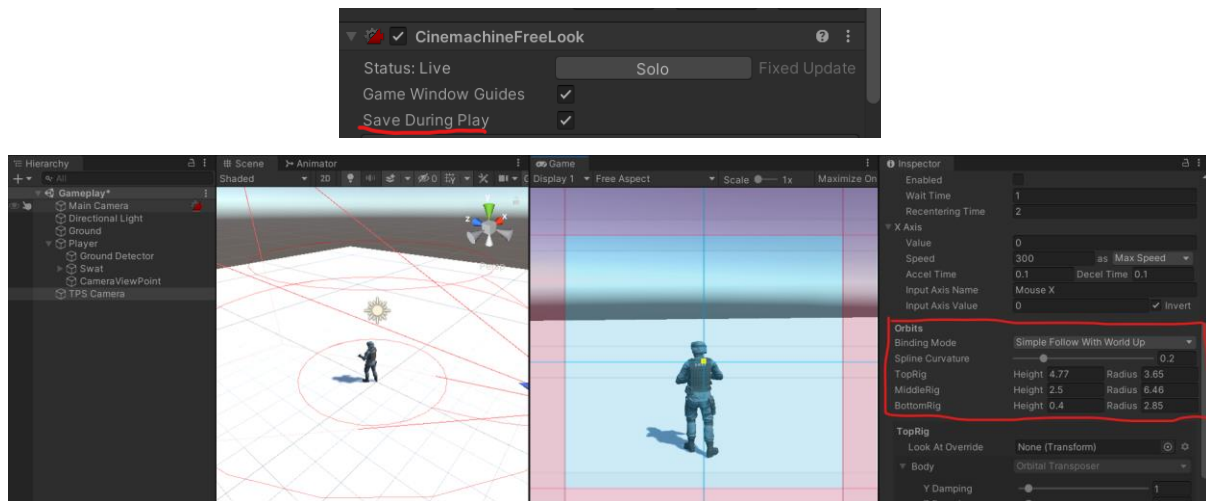
6. Konfigurasi Camera Cinemachine (Orbits)

Konfigurasi selanjutnya adalah untuk mengatur **Orbit Camera Cinemachine** agar sesuai seperti yang diinginkan. Garis orbit terlihat pada Scene berwarna merah dengan 3 lingkaran atas, Tengah, dan bawah. **Fungsi dari Orbit** ini adalah sebagai lintasan kamera.



Cara mengatur Orbit Camera, dapat diatur diinspector dibagian bawah. Lakukan kalibrasi kamera pada **Play mode** agar dapat dilakukan pengecekan secara langsung,

jangan lupa **check Save During Play** agar konfigurasi tersimpan saat keluar dari **Play mode**. Atur sesuai dengan keinginan Kalian.



7. Pengaturan Rotasi Karakter Berdasarkan Rotasi Kamera. (TPS Mode)

Jika Konfigurasi kamera sudah selesai, Langkah selanjutnya adalah melakukan konfigurasi agar karakter berputar sesuai dengan arah kamera melihat. Langkah yang dilakukan adalah membuat **Component Script** baru pada GameObject **Main Camera** Bernama **CameraLogic**, lalu buka Script tersebut.

```
© Unity Script | 0 references
public class CameraLogic : MonoBehaviour
{
    public Transform Player;
    public Transform ViewPoint;
    public float RotationSpeed;

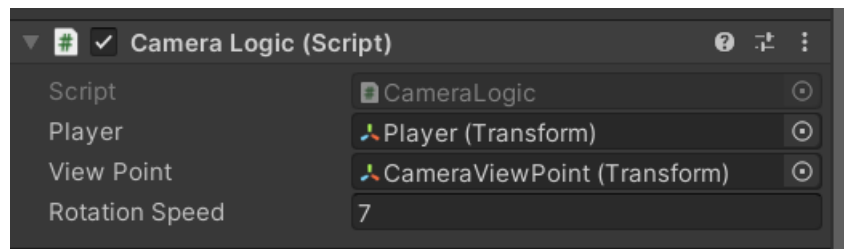
    // Start is called before the first frame update
    © Unity Message | 0 references
    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked;
    }

    // Update is called once per frame
    © Unity Message | 0 references
    private void Update()
    {
        float horizontalInput = Input.GetAxis("Horizontal");
        float verticalInput = Input.GetAxis("Vertical");

        Vector3 viewDir = Player.position - new Vector3(transform.position.x, Player.position.y, transform.position.z);
        ViewPoint.forward = viewDir.normalized;

        Vector3 InputDir = ViewPoint.forward * verticalInput + ViewPoint.right * horizontalInput;
        if (InputDir != Vector3.zero)
        {
            Player.forward = Vector3.Slerp(Player.forward, InputDir.normalized, Time.deltaTime * RotationSpeed);
        }
    }
}
```

Lakukan penulisan Script seperti di atas, jika sudah lakukan deklarasi beberapa variable global pada inspector seperti pada gambar dibawah.

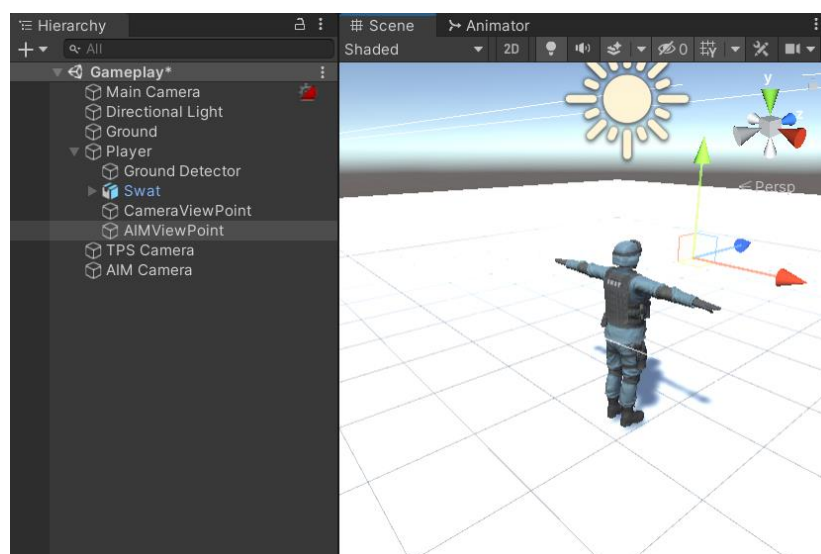


Setelah itu lakukan pengujian apakah karakter sudah dapat berjalan maju sesuai arah kamera melihat atau tidak, jika masih belum bisa, lakukan pengecekan ulang!

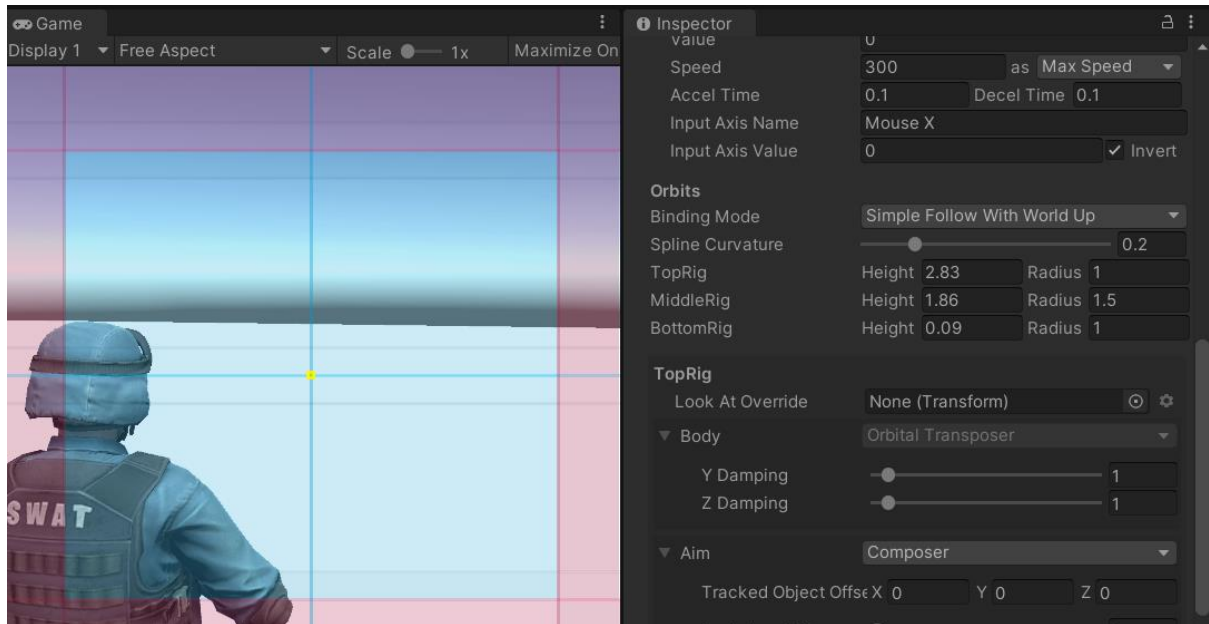
8. Pengaturan Camera AIM Mode.

Proses selanjutnya adalah pembuatan kamera kedua untuk **AIM Mode**, Langkah yang harus dilakukan sama seperti pembuatan kamera **TPS Mode**, yaitu adalah sebagai berikut.

1. Create **FreeLook Camera Cinemachine** rename menjadi **AIM Camera**.
2. Buat **GameObject View Point** Baru untuk AIM Mode Bernama **AIMViewPoint**.
3. Posisikan **AIMViewPoint** di bagian samping kanan kepala **Swat** dan posisikan sedikit maju kedepan. Jika 3 langkah di atas sudah selesai, akan terlihat seperti gambar dibawah.



4. Pada inspector **AIM Camera** Deklarasikan Kolom **Follow** dengan **Swat**, dan Kolom **Look At** dengan **AIMViewPoint**. Pastikan Save During Play Masih tercentang.
5. Selanjutnya masuk kedalam Play Mode, **non aktifkan TPS Camera** agar berpindah ke **AIM Camera**, dan kalibrasikan Orbit **AIM Mode** sesuai seperti kamera saat Membidik. Berikut adalah konfigurasi yang digunakan pada modul ini.



9. Pemrograman AIM Mode.

Selanjutnya adalah proses pemrograman agar kamera dapat berpindah dari TPS mode ke AIM mode, dan juga sebaliknya. Langkah langkahnya adalah sebagai berikut.

1. Tambahkan variable Global pada **CameraLogic**.

```
public Transform Player;
public Transform ViewPoint;
public Transform AIMViewPoint;
public float RotationSpeed;
public GameObject TPSCamera, AIMCamera;
bool TPSMode = true, AIMMode = false;
```

2. Tambahkan beberapa baris kode di method Update.

```
private void Update()
{
    float horizontalInput = Input.GetAxis("Horizontal");
    float verticalInput = Input.GetAxis("Vertical");

    Vector3 viewDir = Player.position - new Vector3(transform.position.x, Player.position.y, transform.position.z);
    ViewPoint.forward = viewDir.normalized;

    if (TPSMode)
    {
        Vector3 InputDir = ViewPoint.forward * verticalInput + ViewPoint.right * horizontalInput;

        if (InputDir != Vector3.zero)
        {
            Player.forward = Vector3.Slerp(Player.forward, InputDir.normalized, Time.deltaTime * RotationSpeed);
        }
    }
    else if (AIMMode)
    {
        Vector3 dirToCombatLookAt = AIMViewPoint.position - new Vector3(transform.position.x, AIMViewPoint.position.y, transform.position.z);
        AIMViewPoint.forward = dirToCombatLookAt.normalized;

        Player.forward = Vector3.Slerp(Player.forward, dirToCombatLookAt.normalized, Time.deltaTime * RotationSpeed);
    }
}
```

3. Lalu tambahkan Method baru untuk mengatur perubahan mode AIM ke TPS di **CameraLogic**.

```
1 reference
public void CameraModeChanger(bool TPS, bool AIM)
{
    TPSMode = TPS;
    AIMMode = AIM;

    if (TPS)
    {
        TPSCamera.SetActive(true);
        AIMCamera.SetActive(false);
    }
    else if (AIM)
    {
        TPSCamera.SetActive(false);
        AIMCamera.SetActive(true);
    }
}
```

4. **Berpindah Pada Script MovementLogic.** Tulis 1 tambahan variable global CameraLogic.

```
private Rigidbody rb;
public float walkspeed, runspeed, jumppower, fallspeed, airMultiplier, HitPoints = 100f;
public Transform PlayerOrientation;
float horizontalInput;
float verticalInput;
Vector3 moveDirection;
bool grounded = true, aerialboost = true;
public Animator anim;
public CameraLogic camlogic;
bool AimMode = false, TPSMode=true;
```

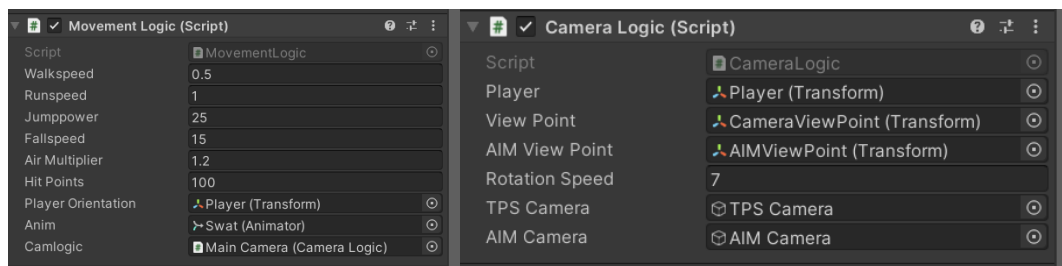
5. Lalu tambahkan 1 baris kode baru pada Method AimModeAdjuster.

```

1 reference
public void AimModeAdjuster()
{
    if (Input.GetKeyDown(KeyCode.Mouse1))
    {
        Debug.Log("mouse1");
        if (AimMode)
        {
            TPSMode = true;
            AimMode = false;
            anim.SetBool("AimMode", false);
        }
        else if (TPSMode)
        {
            TPSMode = false;
            AimMode = true;
            anim.SetBool("AimMode", true);
        }
        camlogic.CameraModeChanger(TPSMode, AimMode);
    }
}

```

6. Terakhir adalah deklarasikan beberapa variable kosong di inspector.



Jika sudah melakukan 6 langkah tersebut, masuk kedalam **Play Mode**, Jika Camera bisa berpindah dari **TPS Camera** Ke **AIM Camera** dan karakter dapat melihat ke arah kamera melihat, maka konfigurasi dan script sudah benar, namun jika belum bisa, lakukan pengecekan ulang!

10. Selesai Untuk Praktikum ke 4

Praktikum 4 dinyatakan selesai jika kamera bisa berganti dari **TPS Camera** ke **AIM Camera**, lalu Ketika **TPS mode** karakter dapat maju sesuai arah kamera dan Ketika **AIM mode** karakter dapat menoleh sesuai arah kamera.

Tugas Modul 4

Tidak ada Tugas Untuk Modul Ke 4

Fokus kepada progressing Game Design Document dan maju secara berkelompok ke dosen pengampu mata kuliah praktikum game.